JavaScript is disabled on your browser.

compact1, compact2, compact3

java.lang.invoke

Class MethodHandles

java.lang.Object
    java.lang.invoke.MethodHandles

```
public class MethodHandles
extends Object
```

This class consists exclusively of static methods that operate on or return method handles. They fall into several categories:

- Lookup methods which help create method handles for methods and fields.
- Combinator methods, which combine or transform pre-existing method handles into new ones.
- Other factory methods to create method handles that emulate other common JVM operations or control flow patterns.

Since:

1.7

Nested Class Summary

Nested Classes

Modifier and Type	Class	Description
static class	MethodHandles.Lookup	A *lookup object* is a factory for creating method handles, when the creation requires access checking.

Method Summary

Concrete Methods

Modifier and Type	Method	Descriptio
static MethodHandle	arrayElementGetter(Class<?> arrayClass)	Produces method handle giv read acce elements an array.
static MethodHandle	arrayElementSetter(Class<?> arrayClass)	Produces method handle giv write acce to elemen an array.
static MethodHandle	catchException(MethodHandle target, Class<? extends Throwable> exType, MethodHandle handler)	Makes a method handle wh adapts a target me handle, by running it inside an exception handler.
static MethodHandle	collectArguments(MethodHandle target, int pos, MethodHandle filter)	Adapts a target me handle by processin sub-seque of its argument with a filt (another method handle).
static MethodHandle	constant(Class<?> type, Object value)	Produces method handle of requested return typ which ret

		the given constant value eve time it is invoked.
static MethodHandle	dropArguments(MethodHandle target, int pos, Class<?>... valueTypes)	Produces method handle wh will disca some dum argument before ca some othe specified *target* me handle.
static MethodHandle	dropArguments(MethodHandle target, int pos, List<Class<?>> valueTypes)	Produces method handle wh will disca some dum argument before ca some othe specified *target* me handle.
static MethodHandle	exactInvoker(MethodType type)	Produces special *invoker method handle* wh can be us to invoke method handle of given type if by **invokeEx**
static MethodHandle	explicitCastArguments(MethodHandle target, MethodType newType)	Produces method handle wh adapts the type of th given met

		handle to new type pairwise argument return typ conversio
static **MethodHandle**	**filterArguments**(**MethodHandle** target, int pos, **MethodHandle**... filters)	Adapts a target me handle by processin one or mo of its argument each with own unary filter func and then calling the target wit each pre-processed argument replaced b the result its correspon filter func
static **MethodHandle**	**filterReturnValue**(**MethodHandle** target, **MethodHandle** filter)	Adapts a target me handle by post-processin return val (if any) wi filter (ano method handle).
static **MethodHandle**	**foldArguments**(**MethodHandle** target, **MethodHandle** combiner)	Adapts a target me handle by processin some of it argument and then calling the target wit

		the result the pre-processin inserted i the origin sequence argument
static **MethodHandle**	**guardWithTest**(**MethodHandle** test, **MethodHandle** target, **MethodHandle** fallback)	Makes a method handle wh adapts a target me handle, by guarding with a tes boolean-valued method handle.
static **MethodHandle**	**identity**(**Class**<?> type)	Produces method handle wh returns its sole argu when invo
static **MethodHandle**	**insertArguments**(**MethodHandle** target, int pos, **Object**... values)	Provides a target me handle wi one or mo *bound argument* advance o the metho handle's invocation
static **MethodHandle**	**invoker**(**MethodType** type)	Produces special *invoker method handle* wh can be us to invoke method handle compatibl

		with the g type, as if **invoke**.
static **MethodHandles.Lookup**	**lookup**()	Returns a **lookup object** wi full capabilitie emulate a supported bytecode behaviors the caller.
static **MethodHandle**	**permuteArguments**(**MethodHandle** target, **MethodType** newType, int... reorder)	Produces method handle wh adapts the calling sequence the given method handle to new type, reorderin the argument
static **MethodHandles.Lookup**	**publicLookup**()	Returns a **lookup object** wh is trusted minimally
static <T extends **Member**> T	**reflectAs**(**Class**<T> expected, **MethodHandle** target)	Performs unchecke "crack" of **direct method handle**.
static **MethodHandle**	**spreadInvoker**(**MethodType** type, int leadingArgCount)	Produces method handle wh will invok any metho handle of given typ

		with a giv number o trailing argument replaced single trailing `Object[]` array.
static **MethodHandle**	**throwException**(**Class**<?> returnType, **Class**<? extends **Throwable**> exType)	Produces method handle wh will throw exception the given exType.

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Method Detail

lookup

```
public static MethodHandles.Lookup lookup()
```

Returns a `lookup object` with full capabilities to emulate all supported bytecode behaviors of the caller. These capabilities include private access to the caller. Factory methods on the lookup object can create direct method handles for any member that the caller has access to via bytecodes, including protected and private fields and methods. This lookup object is a *capability* which may be delegated to trusted agents. Do not store it in place where untrusted code can access it.

This method is caller sensitive, which means that it may return different values to different callers.

For any given caller class `C`, the lookup object returned by this call has equivalent capabilities to any lookup object supplied by the JVM to the bootstrap method of an invokedynamic instruction executing in the same caller class `C`.

Returns:

`a lookup object for the caller of this method, with private access`

publicLookup

```
public static MethodHandles.Lookup publicLookup()
```

Returns a `lookup object` which is trusted minimally. It can only be used to create method handles to publicly accessible fields and methods.

As a matter of pure convention, the lookup class of this lookup object will be `Object`.

Discussion: The lookup class can be changed to any other class `C` using an expression of the form `publicLookup().in(C.class)`. Since all classes have equal access to public names, such a change would confer no new access rights. A public lookup object is always subject to security manager checks. Also, it cannot access caller sensitive methods.

Returns:

`a lookup object which is trusted minimally`

reflectAs

```
public static <T extends Member> T reflectAs(Class<T> expected,
                                             MethodHandle target)
```

Performs an unchecked "crack" of a direct method handle. The result is as if the user had obtained a lookup object capable enough to crack the target method handle, called `Lookup.revealDirect` on the target to obtain its symbolic reference, and then called `MethodHandleInfo.reflectAs` to resolve the symbolic reference to a member.

If there is a security manager, its `checkPermission` method is called with a `ReflectPermission("suppressAccessChecks")` permission.

Type Parameters:

`T - the desired type of the result, either Member or a subtype`

Parameters:

`target - a direct method handle to crack into symbolic reference components`

`expected - a class object representing the desired result type T`

Returns:

`a reference to the method, constructor, or field object`

Throws:

`SecurityException - if the caller is not privileged to call setAccessible`

`NullPointerException - if either argument is null`

`IllegalArgumentException - if the target is not a direct method handle`

`ClassCastException - if the member is not of the expected type`

Since:

`1.8`

arrayElementGetter

```
public static MethodHandle arrayElementGetter(Class<?> arrayClass)
                                        throws IllegalArgumentException
```

Produces a method handle giving read access to elements of an array. The type of the method handle will have a return type of the array's element type. Its first argument will be the array type, and the second will be `int`.

Parameters:

`arrayClass - an array type`

Returns:

`a method handle which can load values from the given array type`

Throws:

`NullPointerException - if the argument is null`

IllegalArgumentException - if arrayClass is not an array type

arrayElementSetter

```
public static MethodHandle arrayElementSetter(Class<?> arrayClass)
                                        throws IllegalArgumentException
```

Produces a method handle giving write access to elements of an array. The type of the method handle will have a void return type. Its last argument will be the array's element type. The first and second arguments will be the array type and int.

Parameters:

arrayClass - the class of an array

Returns:

a method handle which can store values into the array type

Throws:

NullPointerException - if the argument is null

IllegalArgumentException - if arrayClass is not an array type

spreadInvoker

```
public static MethodHandle spreadInvoker(MethodType type,
                                         int leadingArgCount)
```

Produces a method handle which will invoke any method handle of the given `type`, with a given number of trailing arguments replaced by a single trailing `Object[]` array. The resulting invoker will be a method handle with the following arguments:

- a single `MethodHandle` target
- zero or more leading values (counted by `leadingArgCount`)
- an `Object[]` array containing trailing arguments

The invoker will invoke its target like a call to `invoke` with the indicated `type`. That is, if the target is exactly of the given `type`, it will behave like `invokeExact`; otherwise it behave as if `asType` is used to convert the target to the required `type`.

The type of the returned invoker will not be the given `type`, but rather will have all parameters except the first `leadingArgCount` replaced by a single array of type `Object[]`, which will be the final parameter.

Before invoking its target, the invoker will spread the final array, apply reference casts as necessary, and unbox and widen primitive arguments. If, when the invoker is called, the supplied array argument does not have the

correct number of elements, the invoker will throw an `IllegalArgumentException` instead of invoking the target.

This method is equivalent to the following code (though it may be more efficient):

```
MethodHandle invoker = MethodHandles.invoker(type);
int spreadArgCount = type.parameterCount() - leadingArgCount;
invoker = invoker.asSpreader(Object[].class, spreadArgCount);
return invoker;
```

This method throws no reflective or security exceptions.

Parameters:

`type - the desired target type`

`leadingArgCount - number of fixed arguments, to be passed unchanged to the target`

Returns:

`a method handle suitable for invoking any method handle of the given type`

Throws:

`NullPointerException - if type is null`

`IllegalArgumentException - if leadingArgCount is not in the range from 0 to type.parameterCount() inclusive, or if the resulting method handle's type would have too many parameters`

exactInvoker

`public static MethodHandle exactInvoker(MethodType type)`

Produces a special *invoker method handle* which can be used to invoke any method handle of the given type, as if by `invokeExact`. The resulting invoker will have a type which is exactly equal to the desired type, except that it will accept an additional leading argument of type `MethodHandle`.

This method is equivalent to the following code (though it may be more efficient): `publicLookup().findVirtual(MethodHandle.class, "invokeExact", type)`

Discussion: Invoker method handles can be useful when working with variable method handles of unknown types. For example, to emulate an `invokeExact` call to a variable method handle `M`, extract its type `T`, look up the invoker method `X` for `T`, and call the invoker method, as `X.invoke(T, A...)`. (It would not work to call `X.invokeExact`, since the type `T` is unknown.) If spreading, collecting, or other argument transformations

are required, they can be applied once to the invoker `X` and reused on many `M` method handle values, as long as they are compatible with the type of `X`.

(Note: The invoker method is not available via the Core Reflection API. An attempt to call java.lang.reflect.Method.invoke on the declared `invokeExact` or `invoke` method will raise an `UnsupportedOperationException`.)

This method throws no reflective or security exceptions.

Parameters:

`type - the desired target type`

Returns:

`a method handle suitable for invoking any method handle of the given type`

Throws:

`IllegalArgumentException - if the resulting method handle's type would have too many parameters`

invoker

```
public static MethodHandle invoker(MethodType type)
```

Produces a special *invoker method handle* which can be used to invoke any method handle compatible with the given type, as if by `invoke`. The resulting invoker will have a type which is exactly equal to the desired type, except that it will accept an additional leading argument of type `MethodHandle`.

Before invoking its target, if the target differs from the expected type, the invoker will apply reference casts as necessary and box, unbox, or widen primitive values, as if by `asType`. Similarly, the return value will be converted as necessary. If the target is a variable arity method handle, the required arity conversion will be made, again as if by `asType`.

This method is equivalent to the following code (though it may be more efficient): `publicLookup().findVirtual(MethodHandle.class, "invoke", type)`

Discussion: A general method type is one which mentions only `Object` arguments and return values. An invoker for such a type is capable of calling any method handle of the same arity as the general type.

(Note: The invoker method is not available via the Core Reflection API. An attempt to call java.lang.reflect.Method.invoke on the declared `invokeExact` or `invoke` method will raise an `UnsupportedOperationException`.)

This method throws no reflective or security exceptions.

Parameters:

`type - the desired target type`

Returns:

a method handle suitable for invoking any method handle convertible to the given type

Throws:

`IllegalArgumentException` - if the resulting method handle's type would have too many parameters

explicitCastArguments

```
public static MethodHandle explicitCastArguments(MethodHandle target,
                                                 MethodType newType)
```

Produces a method handle which adapts the type of the given method handle to a new type by pairwise argument and return type conversion. The original type and new type must have the same number of arguments. The resulting method handle is guaranteed to report a type which is equal to the desired new type.

If the original type and new type are equal, returns target.

The same conversions are allowed as for `MethodHandle.asType`, and some additional conversions are also applied if those conversions fail. Given types *T0*, *T1*, one of the following conversions is applied if possible, before or instead of any conversions done by `asType`:

- If *T0* and *T1* are references, and *T1* is an interface type, then the value of type *T0* is passed as a *T1* without a cast. (This treatment of interfaces follows the usage of the bytecode verifier.)
- If *T0* is boolean and *T1* is another primitive, the boolean is converted to a byte value, 1 for true, 0 for false. (This treatment follows the usage of the bytecode verifier.)
- If *T1* is boolean and *T0* is another primitive, *T0* is converted to byte via Java casting conversion (JLS 5.5), and the low order bit of the result is tested, as if by `(x & 1) != 0`.
- If *T0* and *T1* are primitives other than boolean, then a Java casting conversion (JLS 5.5) is applied. (Specifically, *T0* will convert to *T1* by widening and/or narrowing.)
- If *T0* is a reference and *T1* a primitive, an unboxing conversion will be applied at runtime, possibly followed by a Java casting conversion (JLS 5.5) on the primitive value, possibly followed by a conversion from byte to boolean by testing the low-order bit.
- If *T0* is a reference and *T1* a primitive, and if the reference is null at runtime, a zero value is introduced.

Parameters:

`target` - the method handle to invoke after arguments are retyped

`newType` - the expected type of the new method handle

Returns:

a method handle which delegates to the target after performing any necessary argument conversions, and arranges for any necessary return value conversions

Throws:

`NullPointerException` - if either argument is null

`WrongMethodTypeException` - if the conversion cannot be made

See Also:

`MethodHandle.asType(java.lang.invoke.MethodType)`

permuteArguments

```
public static MethodHandle permuteArguments(MethodHandle target,
                                            MethodType newType,
                                            int... reorder)
```

Produces a method handle which adapts the calling sequence of the given method handle to a new type, by reordering the arguments. The resulting method handle is guaranteed to report a type which is equal to the desired new type.

The given array controls the reordering. Call `#I` the number of incoming parameters (the value `newType.parameterCount()`, and call `#O` the number of outgoing parameters (the value `target.type().parameterCount()`). Then the length of the reordering array must be `#O`, and each element must be a non-negative number less than `#I`. For every `N` less than `#O`, the `N`-th outgoing argument will be taken from the `I`-th incoming argument, where `I` is `reorder[N]`.

No argument or return value conversions are applied. The type of each incoming argument, as determined by `newType`, must be identical to the type of the corresponding outgoing parameter or parameters in the target method handle. The return type of `newType` must be identical to the return type of the original target.

The reordering array need not specify an actual permutation. An incoming argument will be duplicated if its index appears more than once in the array, and an incoming argument will be dropped if its index does not appear in the array. As in the case of `dropArguments`, incoming arguments which are not mentioned in the reordering array are may be any type, as determined only by `newType`.

```
    import static java.lang.invoke.MethodHandles.*;
    import static java.lang.invoke.MethodType.*;
    ...
    MethodType intfn1 = methodType(int.class, int.class);
```

```
MethodType intfn2 = methodType(int.class, int.class, int.class);
MethodHandle sub = ... (int x, int y) -> (x-y) ...;
assert(sub.type().equals(intfn2));
MethodHandle sub1 = permuteArguments(sub, intfn2, 0, 1);
MethodHandle rsub = permuteArguments(sub, intfn2, 1, 0);
assert((int)rsub.invokeExact(1, 100) == 99);
MethodHandle add = ... (int x, int y) -> (x+y) ...;
assert(add.type().equals(intfn2));
MethodHandle twice = permuteArguments(add, intfn1, 0, 0);
assert(twice.type().equals(intfn1));
assert((int)twice.invokeExact(21) == 42);
```

Parameters:

target - the method handle to invoke after arguments are reordered

newType - the expected type of the new method handle

reorder - an index array which controls the reordering

Returns:

a method handle which delegates to the target after it drops unused arguments and moves and/or duplicates the other arguments

Throws:

NullPointerException - if any argument is null

IllegalArgumentException - if the index array length is not equal to the arity of the target, or if any index array element not a valid index for a parameter of newType, or if two corresponding parameter types in target.type() and newType are not identical,

constant

```
public static MethodHandle constant(Class<?> type,
                                    Object value)
```

Produces a method handle of the requested return type which returns the given constant value every time it is invoked.

Before the method handle is returned, the passed-in value is converted to the requested type. If the requested type is primitive, widening primitive conversions are attempted, else reference conversions are attempted.

The returned method handle is equivalent to identity(type).bindTo(value).

Parameters:

type - the return type of the desired method handle

value - the value to return

Returns:

a method handle of the given return type and no arguments, which always returns the given value

Throws:

NullPointerException - if the type argument is null

ClassCastException - if the value cannot be converted to the required return type

IllegalArgumentException - if the given type is void.class

identity

```
public static MethodHandle identity(Class<?> type)
```

Produces a method handle which returns its sole argument when invoked.

Parameters:

type - the type of the sole parameter and return value of the desired method handle

Returns:

a unary method handle which accepts and returns the given type

Throws:

NullPointerException - if the argument is null

IllegalArgumentException - if the given type is void.class

insertArguments

```
public static MethodHandle insertArguments(MethodHandle target,
                                           int pos,
                                           Object... values)
```

Provides a target method handle with one or more *bound arguments* in advance of the method handle's invocation. The formal parameters to the target corresponding to the bound arguments are called *bound parameters*. Returns a new method handle which saves away the bound arguments. When it is invoked, it receives arguments for any non-bound parameters, binds the saved arguments to their corresponding parameters, and calls the original target.

The type of the new method handle will drop the types for the bound parameters from the original target type, since the new method handle will no longer require those arguments to be supplied by its callers.

Each given argument object must match the corresponding bound parameter type. If a bound parameter type is a primitive, the argument object must be a wrapper, and will be unboxed to produce the primitive value.

The `pos` argument selects which parameters are to be bound. It may range between zero and *N-L* (inclusively), where *N* is the arity of the target method handle and *L* is the length of the values array.

Parameters:

`target - the method handle to invoke after the argument is inserted`

`pos - where to insert the argument (zero for the first)`

`values - the series of arguments to insert`

Returns:

`a method handle which inserts an additional argument, before calling the original method handle`

Throws:

`NullPointerException - if the target or the values array is null`

See Also:

`MethodHandle.bindTo(java.lang.Object)`

dropArguments

```
public static MethodHandle dropArguments(MethodHandle target,
                                         int pos,
                                         List<Class<?>> valueTypes)
```

Produces a method handle which will discard some dummy arguments before calling some other specified *target* method handle. The type of the new method handle will be the same as the target's type, except it will also include the dummy argument types, at some given position.

The `pos` argument may range between zero and *N*, where *N* is the arity of the target. If `pos` is zero, the dummy arguments will precede the target's real arguments; if `pos` is *N* they will come after.

Example:

```
import static java.lang.invoke.MethodHandles.*;
import static java.lang.invoke.MethodType.*;
...
MethodHandle cat = lookup().findVirtual(String.class,
  "concat", methodType(String.class, String.class));
assertEquals("xy", (String) cat.invokeExact("x", "y"));
MethodType bigType = cat.type().insertParameterTypes(0, int.class, String.cl
```

```
    MethodHandle d0 = dropArguments(cat, 0, bigType.parameterList().subList(0,2)
    assertEquals(bigType, d0.type());
    assertEquals("yz", (String) d0.invokeExact(123, "x", "y", "z"));
```

This method is also equivalent to the following code:

```
dropArguments (target, pos, valueTypes.toArray(new Class[0]))
```

Parameters:

`target` - the method handle to invoke after the arguments are dropped

`valueTypes` - the type(s) of the argument(s) to drop

`pos` - position of first argument to drop (zero for the leftmost)

Returns:

a method handle which drops arguments of the given types, before calling the original method handle

Throws:

`NullPointerException` - if the target is null, or if the valueTypes list or any of its elements is null

`IllegalArgumentException` - if any element of valueTypes is void.class, or if pos is negative or greater than the arity of the target, or if the new method handle's type would have too many parameters

dropArguments

```
public static MethodHandle dropArguments(MethodHandle target,
                                         int pos,
                                         Class<?>... valueTypes)
```

Produces a method handle which will discard some dummy arguments before calling some other specified *target* method handle. The type of the new method handle will be the same as the target's type, except it will also include the dummy argument types, at some given position.

The `pos` argument may range between zero and *N*, where *N* is the arity of the target. If `pos` is zero, the dummy arguments will precede the target's real arguments; if `pos` is *N* they will come after.

Example:

```
    import static java.lang.invoke.MethodHandles.*;
    import static java.lang.invoke.MethodType.*;
    ...
```

```
MethodHandle cat = lookup().findVirtual(String.class,
  "concat", methodType(String.class, String.class));
assertEquals("xy", (String) cat.invokeExact("x", "y"));
MethodHandle d0 = dropArguments(cat, 0, String.class);
assertEquals("yz", (String) d0.invokeExact("x", "y", "z"));
MethodHandle d1 = dropArguments(cat, 1, String.class);
assertEquals("xz", (String) d1.invokeExact("x", "y", "z"));
MethodHandle d2 = dropArguments(cat, 2, String.class);
assertEquals("xy", (String) d2.invokeExact("x", "y", "z"));
MethodHandle d12 = dropArguments(cat, 1, int.class, boolean.class);
assertEquals("xz", (String) d12.invokeExact("x", 12, true, "z"));
```

This method is also equivalent to the following code:

```
dropArguments (target, pos, Arrays.asList(valueTypes))
```

Parameters:

target - the method handle to invoke after the arguments are dropped

valueTypes - the type(s) of the argument(s) to drop

pos - position of first argument to drop (zero for the leftmost)

Returns:

a method handle which drops arguments of the given types, before calling the original method handle

Throws:

NullPointerException - if the target is null, or if the valueTypes array or any of its elements is null

IllegalArgumentException - if any element of valueTypes is void.class, or if pos is negative or greater than the arity of the target, or if the new method handle's type would have too many parameters

filterArguments

```
public static MethodHandle filterArguments(MethodHandle target,
                                           int pos,
                                           MethodHandle... filters)
```

Adapts a target method handle by pre-processing one or more of its arguments, each with its own unary filter function, and then calling the target with each pre-processed argument replaced by the result of its corresponding filter function.

The pre-processing is performed by one or more method handles, specified in the elements of the `filters` array. The first element of the filter array corresponds to the `pos` argument of the target, and so on in sequence.

Null arguments in the array are treated as identity functions, and the corresponding arguments left unchanged. (If there are no non-null elements in the array, the original target is returned.) Each filter is applied to the corresponding argument of the adapter.

If a filter `F` applies to the `N`th argument of the target, then `F` must be a method handle which takes exactly one argument. The type of `F`'s sole argument replaces the corresponding argument type of the target in the resulting adapted method handle. The return type of `F` must be identical to the corresponding parameter type of the target.

It is an error if there are elements of `filters` (null or not) which do not correspond to argument positions in the target.

Example:

```
import static java.lang.invoke.MethodHandles.*;
import static java.lang.invoke.MethodType.*;
...
MethodHandle cat = lookup().findVirtual(String.class,
  "concat", methodType(String.class, String.class));
MethodHandle upcase = lookup().findVirtual(String.class,
  "toUpperCase", methodType(String.class));
assertEquals("xy", (String) cat.invokeExact("x", "y"));
MethodHandle f0 = filterArguments(cat, 0, upcase);
assertEquals("Xy", (String) f0.invokeExact("x", "y")); // Xy
MethodHandle f1 = filterArguments(cat, 1, upcase);
assertEquals("xY", (String) f1.invokeExact("x", "y")); // xY
MethodHandle f2 = filterArguments(cat, 0, upcase, upcase);
assertEquals("XY", (String) f2.invokeExact("x", "y")); // XY
```

Here is pseudocode for the resulting adapter:

```
V target(P... p, A[i]... a[i], B... b);
A[i] filter[i](V[i]);
T adapter(P... p, V[i]... v[i], B... b) {
  return target(p..., f[i](v[i])..., b...);
}
```

Parameters:

`target` - the method handle to invoke after arguments are filtered

`pos` - the position of the first argument to filter

filters - method handles to call initially on filtered arguments

Returns:

method handle which incorporates the specified argument filtering logic

Throws:

NullPointerException - if the target is null or if the filters array is null

IllegalArgumentException - if a non-null element of filters does not match a corresponding argument type of target as described above, or if the pos+filters.length is greater than target.type().parameterCount(), or if the resulting method handle's type would have too many parameters

collectArguments

```
public static MethodHandle collectArguments(MethodHandle target,
                                            int pos,
                                            MethodHandle filter)
```

Adapts a target method handle by pre-processing a sub-sequence of its arguments with a filter (another method handle). The pre-processed arguments are replaced by the result (if any) of the filter function. The target is then called on the modified (usually shortened) argument list.

If the filter returns a value, the target must accept that value as its argument in position `pos`, preceded and/or followed by any arguments not passed to the filter. If the filter returns void, the target must accept all arguments not passed to the filter. No arguments are reordered, and a result returned from the filter replaces (in order) the whole subsequence of arguments originally passed to the adapter.

The argument types (if any) of the filter replace zero or one argument types of the target, at position `pos`, in the resulting adapted method handle. The return type of the filter (if any) must be identical to the argument type of the target at position `pos`, and that target argument is supplied by the return value of the filter.

In all cases, `pos` must be greater than or equal to zero, and `pos` must also be less than or equal to the target's arity.

Example:

```
import static java.lang.invoke.MethodHandles.*;
import static java.lang.invoke.MethodType.*;
...
MethodHandle deepToString = publicLookup()
  .findStatic(Arrays.class, "deepToString", methodType(String.class, Object[
```

```
MethodHandle ts1 = deepToString.asCollector(String[].class, 1);
assertEquals("[strange]", (String) ts1.invokeExact("strange"));

MethodHandle ts2 = deepToString.asCollector(String[].class, 2);
assertEquals("[up, down]", (String) ts2.invokeExact("up", "down"));

MethodHandle ts3 = deepToString.asCollector(String[].class, 3);
MethodHandle ts3_ts2 = collectArguments(ts3, 1, ts2);
assertEquals("[top, [up, down], strange]",
             (String) ts3_ts2.invokeExact("top", "up", "down", "strange"));

MethodHandle ts3_ts2_ts1 = collectArguments(ts3_ts2, 3, ts1);
assertEquals("[top, [up, down], [strange]]",
             (String) ts3_ts2_ts1.invokeExact("top", "up", "down", "strange"

MethodHandle ts3_ts2_ts3 = collectArguments(ts3_ts2, 1, ts3);
assertEquals("[top, [[up, down, strange], charm], bottom]",
             (String) ts3_ts2_ts3.invokeExact("top", "up", "down", "strange"
```

Here is pseudocode for the resulting adapter:

```
T target(A...,V,C...);
V filter(B...);
T adapter(A... a,B... b,C... c) {
  V v = filter(b...);
  return target(a...,v,c...);
}
// and if the filter has no arguments:
T target2(A...,V,C...);
V filter2();
T adapter2(A... a,C... c) {
  V v = filter2();
  return target2(a...,v,c...);
}
// and if the filter has a void return:
T target3(A...,C...);
void filter3(B...);
void adapter3(A... a,B... b,C... c) {
  filter3(b...);
  return target3(a...,c...);
}
```

A collection adapter `collectArguments(mh, 0, coll)` is equivalent to one which first "folds" the affected arguments, and then drops them, in separate steps as follows:

```
mh = MethodHandles.dropArguments(mh, 1, coll.type().parameterList()); //ste
mh = MethodHandles.foldArguments(mh, coll); //step 1
```

If the target method handle consumes no arguments besides than the result (if any) of the filter `coll`, then `collectArguments(mh, 0, coll)` is equivalent to `filterReturnValue(coll, mh)`. If the filter method handle `coll` consumes one argument and produces a non-void result, then `collectArguments(mh, N, coll)` is equivalent to `filterArguments(mh, N, coll)`. Other equivalences are possible but would require argument permutation.

Parameters:

`target` - the method handle to invoke after filtering the subsequence of arguments

`pos` - the position of the first adapter argument to pass to the filter, and/or the target argument which receives the result of the filter

`filter` - method handle to call on the subsequence of arguments

Returns:

method handle which incorporates the specified argument subsequence filtering logic

Throws:

`NullPointerException` - if either argument is null

`IllegalArgumentException` - if the return type of filter is non-void and is not the same as the pos argument of the target, or if pos is not between 0 and the target's arity, inclusive, or if the resulting method handle's type would have too many parameters

See Also:

`foldArguments(java.lang.invoke.MethodHandle, java.lang.invoke.MethodHandle)`, `filterArguments(java.lang.invoke.MethodHandle, int, java.lang.invoke.MethodHandle...)`, `filterReturnValue(java.lang.invoke.MethodHandle, java.lang.invoke.MethodHandle)`

filterReturnValue

```
public static MethodHandle filterReturnValue(MethodHandle target,
                                             MethodHandle filter)
```

Adapts a target method handle by post-processing its return value (if any) with a filter (another method handle). The result of the filter is returned from the adapter.

If the target returns a value, the filter must accept that value as its only argument. If the target returns void, the filter must accept no arguments.

The return type of the filter replaces the return type of the target in the resulting adapted method handle. The argument type of the filter (if any) must be identical to the return type of the target.

Example:

```
import static java.lang.invoke.MethodHandles.*;
import static java.lang.invoke.MethodType.*;
...
MethodHandle cat = lookup().findVirtual(String.class,
  "concat", methodType(String.class, String.class));
MethodHandle length = lookup().findVirtual(String.class,
  "length", methodType(int.class));
System.out.println((String) cat.invokeExact("x", "y")); // xy
MethodHandle f0 = filterReturnValue(cat, length);
System.out.println((int) f0.invokeExact("x", "y")); // 2
```

Here is pseudocode for the resulting adapter:

```
 V target(A...);
 T filter(V);
 T adapter(A... a) {
   V v = target(a...);
   return filter(v);
 }
 // and if the target has a void return:
 void target2(A...);
 T filter2();
 T adapter2(A... a) {
   target2(a...);
   return filter2();
 }
 // and if the filter has a void return:
 V target3(A...);
 void filter3(V);
 void adapter3(A... a) {
   V v = target3(a...);
   filter3(v);
 }
```

Parameters:

`target - the method handle to invoke before filtering the return value`

filter - method handle to call on the return value

Returns:

method handle which incorporates the specified return value filtering logic

Throws:

NullPointerException - if either argument is null

IllegalArgumentException - if the argument list of filter does not match the return type of target as described above

foldArguments

```
public static MethodHandle foldArguments(MethodHandle target,
                                         MethodHandle combiner)
```

Adapts a target method handle by pre-processing some of its arguments, and then calling the target with the result of the pre-processing, inserted into the original sequence of arguments.

The pre-processing is performed by `combiner`, a second method handle. Of the arguments passed to the adapter, the first `N` arguments are copied to the combiner, which is then called. (Here, `N` is defined as the parameter count of the combiner.) After this, control passes to the target, with any result from the combiner inserted before the original `N` incoming arguments.

If the combiner returns a value, the first parameter type of the target must be identical with the return type of the combiner, and the next `N` parameter types of the target must exactly match the parameters of the combiner.

If the combiner has a void return, no result will be inserted, and the first `N` parameter types of the target must exactly match the parameters of the combiner.

The resulting adapter is the same type as the target, except that the first parameter type is dropped, if it corresponds to the result of the combiner.

(Note that `dropArguments` can be used to remove any arguments that either the combiner or the target does not wish to receive. If some of the incoming arguments are destined only for the combiner, consider using `asCollector` instead, since those arguments will not need to be live on the stack on entry to the target.)

Example:

```
import static java.lang.invoke.MethodHandles.*;
import static java.lang.invoke.MethodType.*;
...
MethodHandle trace = publicLookup().findVirtual(java.io.PrintStream.class,
```

```
      "println", methodType(void.class, String.class))
        .bindTo(System.out);
    MethodHandle cat = lookup().findVirtual(String.class,
      "concat", methodType(String.class, String.class));
    assertEquals("boojum", (String) cat.invokeExact("boo", "jum"));
    MethodHandle catTrace = foldArguments(cat, trace);
    // also prints "boo":
    assertEquals("boojum", (String) catTrace.invokeExact("boo", "jum"));
```

Here is pseudocode for the resulting adapter:

```
    // there are N arguments in A...
    T target(V, A[N]..., B...);
    V combiner(A...);
    T adapter(A... a, B... b) {
      V v = combiner(a...);
      return target(v, a..., b...);
    }
    // and if the combiner has a void return:
    T target2(A[N]..., B...);
    void combiner2(A...);
    T adapter2(A... a, B... b) {
      combiner2(a...);
      return target2(a..., b...);
    }
```

Parameters:

target - the method handle to invoke after arguments are combined

combiner - method handle to call initially on the incoming arguments

Returns:

method handle which incorporates the specified argument folding logic

Throws:

NullPointerException - if either argument is null

IllegalArgumentException - if combiner's return type is non-void and not the same as the first argument type of the target, or if the initial N argument types of the target (skipping one matching the combiner's return type) are not identical with the argument types of combiner

guardWithTest

```
public static MethodHandle guardWithTest(MethodHandle test,
                                         MethodHandle target,
                                         MethodHandle fallback)
```

Makes a method handle which adapts a target method handle, by guarding it with a test, a boolean-valued method handle. If the guard fails, a fallback handle is called instead. All three method handles must have the same corresponding argument and return types, except that the return type of the test must be boolean, and the test is allowed to have fewer arguments than the other two method handles.

Here is pseudocode for the resulting adapter:

```
boolean test(A...);
T target(A...,B...);
T fallback(A...,B...);
T adapter(A... a,B... b) {
  if (test(a...))
    return target(a..., b...);
  else
    return fallback(a..., b...);
}
```

Note that the test arguments (`a...` in the pseudocode) cannot be modified by execution of the test, and so are passed unchanged from the caller to the target or fallback as appropriate.

Parameters:

`test` - method handle used for test, must return boolean

`target` - method handle to call if test passes

`fallback` - method handle to call if test fails

Returns:

method handle which incorporates the specified if/then/else logic

Throws:

`NullPointerException` - if any argument is null

`IllegalArgumentException` - if test does not return boolean, or if all three method types do not match (with the return type of test changed to match that of the target).

catchException

```
public static MethodHandle catchException(MethodHandle target,
                                          Class<? extends Throwable> exType,
                                          MethodHandle handler)
```

Makes a method handle which adapts a target method handle, by running it inside an exception handler. If the target returns normally, the adapter returns that value. If an exception matching the specified type is thrown, the fallback handle is called instead on the exception, plus the original arguments.

The target and handler must have the same corresponding argument and return types, except that handler may omit trailing arguments (similarly to the predicate in `guardWithTest`). Also, the handler must have an extra leading parameter of `exType` or a supertype.

Here is pseudocode for the resulting adapter:

```
T target(A..., B...);
T handler(ExType, A...);
T adapter(A... a, B... b) {
  try {
    return target(a..., b...);
  } catch (ExType ex) {
    return handler(ex, a...);
  }
}
```

Note that the saved arguments (`a...` in the pseudocode) cannot be modified by execution of the target, and so are passed unchanged from the caller to the handler, if the handler is invoked.

The target and handler must return the same type, even if the handler always throws. (This might happen, for instance, because the handler is simulating a `finally` clause). To create such a throwing handler, compose the handler creation logic with `throwException`, in order to create a method handle of the correct return type.

Parameters:

`target` - method handle to call

`exType` - the type of exception which the handler will catch

`handler` - method handle to call if a matching exception is thrown

Returns:

method handle which incorporates the specified try/catch logic

Throws:

NullPointerException - if any argument is null

IllegalArgumentException - if handler does not accept the given exception type, or if the method handle types do not match in their return types and their corresponding parameters

throwException

```
public static MethodHandle throwException(Class<?> returnType,
                                          Class<? extends Throwable> exType)
```

Produces a method handle which will throw exceptions of the given exType. The method handle will accept a single argument of exType, and immediately throw it as an exception. The method type will nominally specify a return of returnType. The return type may be anything convenient: It doesn't matter to the method handle's behavior, since it will never return normally.

Parameters:

returnType - the return type of the desired method handle

exType - the parameter type of the desired method handle

Returns:

method handle which can throw the given exceptions

Throws:

NullPointerException - if either argument is null

Submit a bug or feature

For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.

Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.